

BOTBOND · BONDED AGENT ACCESS

API key를 기다리는 동안 실시간 기회는 사라집니다

BotBond는 공급자가 직접 설치하는 에이전트 접근 게이트웨이이다. 처음 보는 에이전트가 할 일을 설명하면, 공급자는 필요한 API 권한·호출 한도·비용만 담은 짧은 세션을 연다.

예약과 주문처럼 비용이 뒤늦게 생기는 작업은 환불 가능한 온체인 담보로 관리한다.

Permissionless agent onboarding for structured APIs · Google Cloud × Solana

01 · WHY NOW

에이전트 트래픽은 급증했지만, 접근권은 아직 전부 아니면 전무합니다

53%

전체 인터넷 트래픽에서 봇이 차지하는 비중

40%

전체 트래픽에서 악성 봇이 차지하는 비중

+7,851%

에이전트 AI 트래픽 증가율

공급자는 unknown automation을 막아야 한다. 그러나 가격·재고·견적처럼 바로 결정해야 하는 데이터를 찾는 정상 에이전트까지 함께 막힌다. API key를 발급할 때까지 기다리면 그 요청의 가치는 이미 사라진다.

Sources: Thales, Imperva Bad Bot Report · HUMAN Security, Agentic AI traffic research

BotBond는 이 사이에 **짧고 제한적인 공식 접근 경로**를 만든다.

WAF와 API key 사이에는 세 번째 선택지가 필요합니다

방식	잘하는 일	남는 공백
Cloudflare / WAF	비협조적 트래픽 차단	협조적인 미등록 에이전트의 온보딩
API key	장기 고객 인증·청구	일회성 접근의 가입·심사 비용
Rate limit	호출량 통제	자연어 목적과 endpoint 의미
pay-per-use	데이터 사용료	예약 등 사후 의무의 담보
Web Bot Auth	알려진 에이전트의 신원 확인	미등록 에이전트가 즉시 책임을 보일 방법

WAF는 비협조적 트래픽을 계속 막고, 반복 고객은 API key를 쓴다. BotBond는 그 사이에서 처음 보는 에이전트에게만 목적·비용·시간이 정해진 접근권을 발급한다.

첫 시장은 검색으로 대체할 수 없는 실시간 구조화 API입니다

공급자

가격·재고·견적·예약 API, MCP와 전문 데이터. 일반 경로는 담은 채 새 수요를 유료 세션으로 받는다.

에이전트

구매·조달·여행·운영 에이전트. 가입 대기 대신 지금 필요한 범위와 비용을 먼저 확인한다.

최종 사용자

검색 결과가 아닌 실시간 가격·재고·가능 여부로 바로 판단하고 실행한다.

처음에는 커머스·B2B 조달·MCP 데이터 API에 집중한다. 이후 여행 좌석, 물류 상태, 시장 데이터, 운영 API까지 같은 접근권 모델로 넓힌다.

시장 신호도 있다. TollBit은 약 7,000개 퍼블리셔 사이트의 AI 접근 수익화를 지원하고, RSL 1.0은 1,500개 이상 조직의 지지를 받는다. 다만 BotBond의 첫 고객은 퍼블리셔가 아니라, 요청 하나가 매출·비용·결정으로 이어지는 실시간 API 공급자다.

Sources: TollBit CEO/Series A disclosures · RSL 1.0 announcement. 위 수치는 시장 검증 신호이며 BotBond의 TAM 추정치가 아닙니다.

보안 정책은 유지하고, 에이전트에게만 별도 입구를 엽니다

일반 자동화 요청

```
Agent
→ Cloudflare / WAF
→ 기존 정책 또는 차단
```

기존 보안 정책은 그대로 둔다.

운영자가 연 에이전트 경로

```
Agent
→ WAF allowlisted path
→ BotBond Gateway
→ Scoped Origin API
```

작업과 조건에 동의한 에이전트만 이 경로를 쓴다.

WAF를 건너뛰면 공급자는 보안 정책과 접근 통제권을 잃는다. BotBond가 차단을 풀어 주면 악성 자동화도 같은 문을 쓴다. 그래서 BotBond는 WAF 뒤, 운영자가 허용한 경로에만 붙는다. 이 경로에서만 처음 보는 에이전트가 정해진 조건으로 세션을 연다.

일반 웹/API는 계속 닫아두고, `/.well-known/agent-access` 와 범위가 제한된 에이전트 API만 별도 정책으로 연다.

데모의 최초 403은 **BShop edge policy 재현**이며 실제 Cloudflare zone 이벤트가 아닙니다.

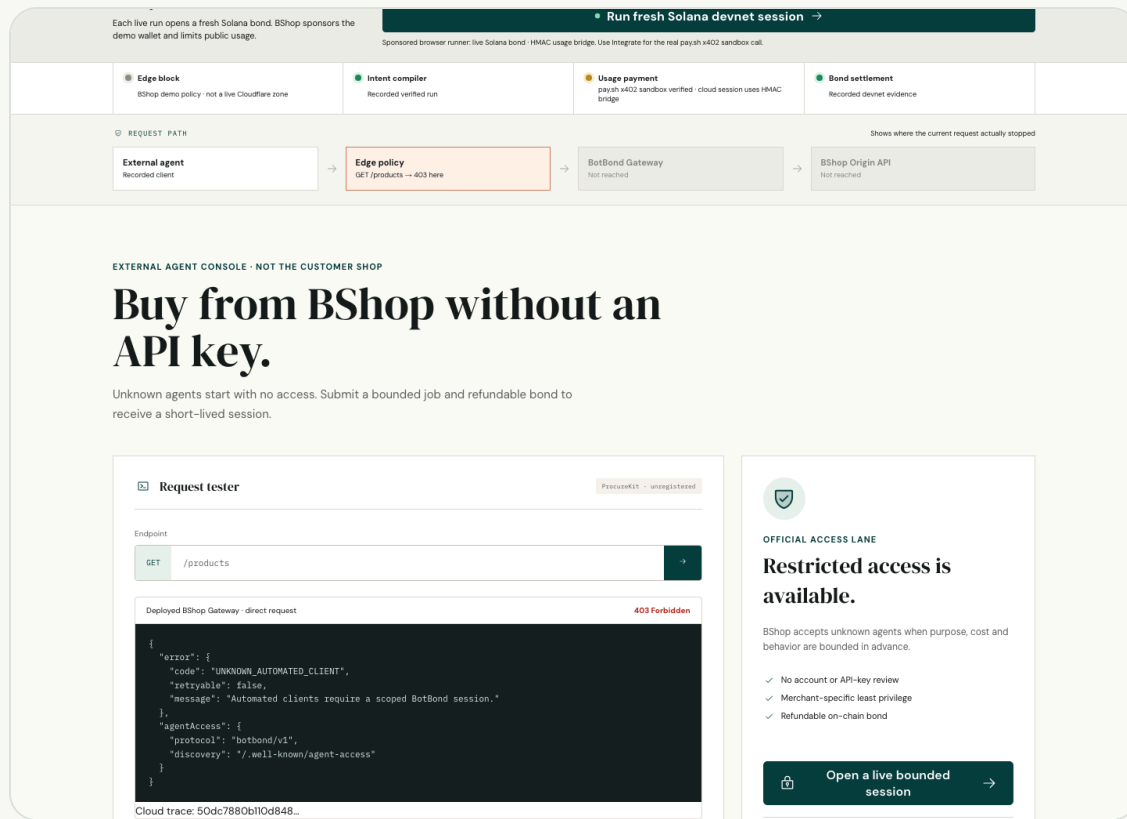
실제 요청도 일반 경로에서 멈춥니다

미등록 agent의 직접 요청

배포된 BShop Gateway는 `GET /products` 를 `403` 으로 돌려보낸다. Origin API에는 요청이 닿지 않는다.

응답에는 우회 방법 대신 공식 discovery 경로 `/.well-known/agent-access` 가 들어 있다. 에이전트는 이 경로에서 조건을 확인한 뒤에만 다음 단계로 간다.

이 캡처는 live Gateway 요청이다. BShop의 edge policy를 보여 주며, Cloudflare zone 로그라고 주장하지 않는다.



직접 요청 `403` · discovery 경로 반환 · origin 미도달

에이전트는 API 구조 대신 할 일을 설명한다

에이전트의 작업 요청

1,500 이하 노트북의 가격과 재고를 비교하고 마지막 한대를 예약한 뒤 해제해줘. 판매자 연락처와 리뷰는 필요 없어.

공급자 정책 초안

```
{
  "allowed": [
    "GET /products",
    "GET /products/{id}/inventory",
    "POST /reservations",
    "POST /reservations/{id}/release"
  ],
  "max_calls": 25,
  "reservation_ttl": "60s"
}
```

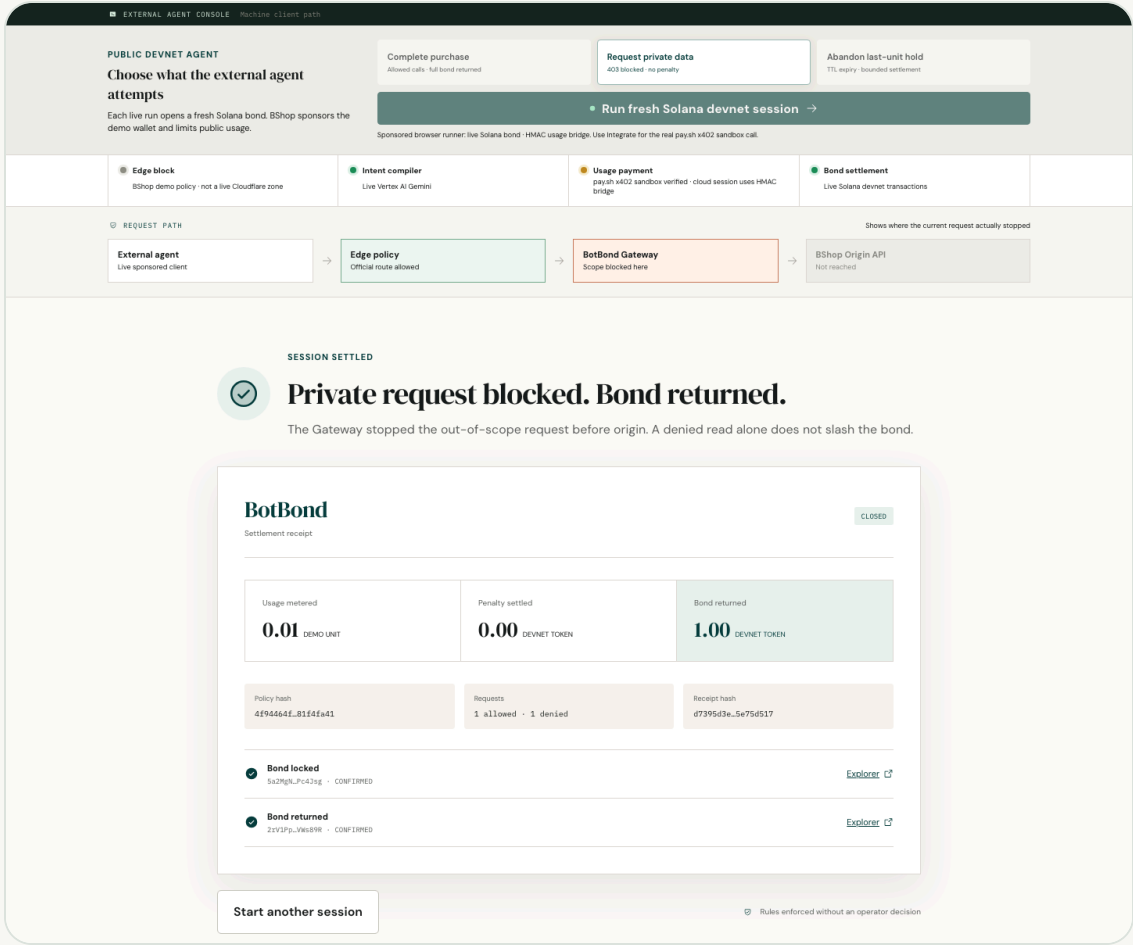
같은 가격·재고 비교라도 상점마다 endpoint와 field 이름이 다르다. 에이전트가 API 구조를 직접 추측하면 공급자 문서를 매번 학습해야 하고, 넓은 권한을 요구하기 쉽다. BotBond는 작업 설명과 공급자의 API catalog를 맞춰 필요한 범위만 담은 정책 초안을 만든다.

Gemini는 정책 초안을 만들고, Gateway가 endpoint·field·rate·TTL을 코드로 집행한다. 세션이 시작된 뒤 AI가 권한을 넓히거나 돈을 정산하지 않는다.

세션 안에서는 약속한 범위만 실행된다

시도	결과	Origin 도달	담보 변화
session 없이 재고 조회	Edge policy 403	아니오	없음
허용된 가격·재고 조회	Gateway 200	예, 허용 field만	없음
seller contacts 요청	Scope 403	아니오	없음
예약 후 정상 해제	Session close	예	전액 반환
예약 후 TTL 방치	Objective expiry	예약만 도달	0.25 제한 정산

차단은 몰수가 아니다. 실제 비용을 만든 bonded action의 객관적 위반만 사전에 서명한 상한 안에서 정산한다.



범위 밖 private request는 Gateway에서 차단되고, bond는 전액 반환됩니다.

요청부터 정산까지 역할을 나눴다

```
Sponsored Browser Agent — HMAC demo bridge ┐
Own-wallet Agent CLI — pay.sh x402 sandbox ┐ ┌ BotBond Gateway · Cloud Run
                                           └ Vertex AI Gemini → policy
                                           └ Firestore → session + evidence
                                           └ deterministic guard → scope / rate / TTL
                                           └ BShop Origin API → price / inventory
                                           └ Solana devnet → bond / refund / settlement
```

브라우저는 Firestore·서명 키에 직접 접근하지 않는다. **pay.sh 결제는 own-wallet CLI만 수행**하고, browser는 HMAC demo bridge를 명시한다. Solana settlement는 confirmed transaction만 Explorer에 연결한다.

지금 동작하는 범위와 데모 경계

구성	상태	검증 증거
Vertex AI Gemini	LIVE	실행마다 새 policy와 hash
Cloud Run + Firestore	LIVE	public API, session state, events
Solana bond program	LIVE DEVNET	open + refund/settlement tx
pay.sh per-call rail	HOSTED SANDBOX	실제 402 → pay → 200
Browser session activation	DEMO BRIDGE	HMAC credential, pay.sh verifier 아님
Cloudflare zone/WAF	NOT INTEGRATED	BShop Gateway가 403 정책만 재현

온체인 자산은 **devnet test mint**이며 실제 USDC가 아닙니다.

브라우저와 외부 에이전트의 결제 경로를 구분한다

Normal web traffic

Agent → Cloudflare WAF → existing site policy

Unknown automation remains blocked.

Merchant-approved agent lane

Agent → WAF allowlisted path → BotBond → scoped API

No Cloudflare bypass. The site opens this route deliberately.

LIVE CONNECTION CHECK

Test BShop's public integration surface.

These are browser requests to deployed Cloud Run services—not fixture events. The final payment is intentionally not made by this browser.

→ Run connection check

Agent discovery 200

GET /.well-known/agent-access

Program EGBxKPV6, published by discovery

Cloud trace 79a78ac254b75d2...

Direct product request 403

GET /products

UNKNOWN_AUTOMATED_CLIENT

Cloud trace 4334cc791a57b96...

pay.sh payment gate 402

GET /v1/access/browser-check/products

x402 payment challenge returned

Cloud trace 79a78ac254b75d2...

⚠ '402' proves that the hosted pay.sh sandbox gate challenged this request. Payment and a scoped '200' require the external-agent command below; this page never claims to pay on the visitor's behalf.

01

Discover

GET /.well-known/agent-access

Read the catalog, session endpoints, Solana program and devnet onboarding mode.

02

Declare intent

POST /v2/intents

```
{
  "task": "Compare price and stock",
  "agentWallet": "<SOLANA_PUBLIC_KEY>",
  "budget": { ... }
}
```

Gemini maps the request to BShop's own endpoint and field catalog.

03

Open the bond

```
npm run example:external-agent -- \
  --gateway https://... \
  --wallet ~/.config/solana/id.json
```

The example creates a devnet test mint, opens the real bond, activates a short session and prints Explorer links.

OWN-WALLET EXECUTION

Run the paid sandbox rail and devnet bond yourself.

Your local Solana keypair signs the bond open. The CLI then performs pay.sh sandbox '402 → payment → scoped 200' before

</> Copy external-agent command

Browser sponsored run

Gemini, Gateway, Firestore와 Solana devnet bond를 실제로 실행한다. 사용량 credential은 HMAC demo bridge이며 pay.sh verifier가 아니다.

Own-wallet agent CLI

자기 Solana 지갑으로 pay.sh hosted sandbox의 402 → payment → scoped 200 을 수행한다. 성공하면 상품 응답과 bond open·정산 Explorer 링크가 출력된다.

서로 다른 rail을 하나의 결제처럼 보이게 만들지 않는다.

공개 connection check: discovery 200 · 직접 요청 403 · pay.sh gate 402

개발자는 discovery에서 세션 발급과 호출을 시작한다

1. 공개 정보 조회

```
GET /.well-known/agent-access
```

intent-session endpoint, payment rail, Solana program ID와 지원 범위를 받는다.

2. 작업과 지갑 제출

```
POST /v1/intents  
{ task, wallet, budget }
```

Gemini가 공급자 catalog 안에서 policy와 policy hash를 만든다. 에이전트는 담보를 열기 전에 조건을 검토한다.

3. 세션으로 호출

```
x-botbond-session-token: <token>  
GET /v1/access/:sessionId/products
```

정책 hash로 devnet bond를 연 뒤 세션을 발급받는다. 허용된 endpoint만 호출할 수 있다.

가장 쉬운 실행 방법은 `/integrate`의 `own-wallet` command다. discovery부터 `pay.sh` sandbox 호출, `bond open·close`, Explorer URL 출력까지 한 번에 실행한다.

정산 결과는 Solana Explorer에서 누구나 확인한다

The screenshot shows the Solana Explorer (BETA) interface. At the top, there's a search bar and navigation links for Feature Gates, Inspector, and Devnet. The main section is titled 'Transaction' and shows a 'Summary' tab. The transaction status is 'Success' and 'Finalized (MAX Confirmations)'. The fee payer is 'GMrS1AR2MhVW9cDmQJ4RApQz6iTS17srfb8bwucJ0a6'. The transaction cost is 15,239 SOL. Below the summary, there's a table for 'Accounts & SQL balance' showing the state of accounts involved in the transaction.

#	ADDRESS	CHANGE (SOL)	POST BALANCE (SOL)
1	GMrS1AR2MhVW9cDmQJ4RApQz6iTS17srfb8bwucJ0a6 Fee Payer Signer Writable	-0.000005	0.09131352
2	7GacpUgWutAFwTTbhoynSv2FeyXVjDg8io7smdBnhoK3 Writable	+0	0.00283928
3	8sW8dsDERhh2JJPt7XbQfn5yegArkZMawFXyoB5qh1ny Writable	+0	0.00283928
4	9sMQjYh9nHZX5qUajMfWmJ6W7J7azwXS11A4Tiepc9d Writable	+0	0.0025752

확인 순서

영수증의 **Bond locked** 또는 **Bond returned** 링크를 열어 **Success / Finalized**, transaction signature, 계정 변화와 BotBond program log를 확인한다.

이번 own-wallet 실행 결과

pay.sh sandbox 결제 후 상품·재고 호출이 성공했다. usage 4000, bond refund 1000000 atomic units가 기록됐다.

Bond open Explorer · Refund Explorer

devnet test mint 기준입니다. session credential은 HMAC demo bridge이며, 실제 USDC 거래나 pay.sh 반복 세션 검증을 뜻하지 않습니다.

3분 안에 정상 접근, 보호, 정산을 차례로 확인한다

0:00-0:40 · 상점과 입구

Shop에서 마지막 한 대의 재고를 확인한다. 이어 **Agent Console**에서 미등록 에이전트의 direct request가 Gateway에도 닿지 않았음을 본다. **Integrate**는 discovery 200, direct 403, pay.sh gate 402 를 나란히 보여 준다.

0:40-1:45 · 정상 세션

Complete purchase를 실행한다. Gemini가 작업 설명을 최소 권한 policy로 만들고, 조건 확인 뒤 fresh Solana devnet bond가 열린다. 허용된 요청은 origin에 도달하고 bond는 전액 반환된다. 새 signature와 Explorer를 바로 연다.

1:45-2:20 · 보호와 제한 정산

Request private data는 Gateway에서 멈추고 origin은 회색으로 남는다. 차단에는 차감이 없다. 이어 **Abandon last-unit hold**는 TTL 뒤 사전 합의한 0.25만 정산하고 나머지 bond를 돌려준다. Merchant Ops에서 영수증을 확인한다.

2:20-3:00 · 외부 에이전트 검증

Integrate의 own-wallet 명령을 실행한다. terminal에서 pay.sh hosted sandbox의 402 → payment → scoped 200 과 fresh bond open·refund signature를 확인한다. browser bridge와 같은 실행 경로로 편집하지 않는다.

심사위원이 직접 확인할 수 있다

라이브 제품

`botbond-bshop.vercel.app`

Shop · Agent · Merchant · Integrate

내 지갑으로 실행

```
npm run example:external-agent -- \
  --gateway https://... \
  --wallet ~/.config/solana/id.json
```

제출물: 발표 PDF · 재현 가능한 GitHub/README · 3분 실제 온체인 영상 · 심사 기간 접근 가능한 endpoint

Program: `EG9r...KaRKR` · asset: devnet test mint, not USDC

처음 보는 에이전트에게 필요한 것은 즉시 신뢰가 아니라, 범위가 분명하고 정산 규칙이 정해진 접근권입니다.